

HW2: Detect AI Generated Text

RNN and Transformer — Comprehensive Report

李捷新

April 2026

Abstract

This report presents a comprehensive study on detecting AI-generated text using BERT-based classifiers. We fine-tune both `bert-base-cased` (108M parameters) and `bert-large-cased` (334M parameters) on the DAIGT V2 Train Dataset (44,868 essays). BERT-large achieves perfect ROC-AUC of 1.0000 and accuracy of 0.9980. We compare against TF-IDF baselines (best AUC: 0.9997 with Linear SVM), conduct extensive ablation studies on learning rate, batch size, sequence length, and training epochs, and perform adversarial attacks using a locally deployed DeepSeek-R1:8B model via Ollama plus programmatic perturbations. The BERT detector proves highly robust: 0/30 single-pass LLM attacks succeed, 1/5 iterative attacks succeed (20%), and only 1/30 programmatic perturbations evade detection (3.3%). All experiments were conducted on an NVIDIA RTX 4090 GPU with fp16 mixed precision.

Contents

1	Data Analysis & Baseline (Part 1)	2
1.1	Exploratory Data Analysis	2
1.2	Baseline Models	3
2	BERT Fine-Tuning & Scaling (Part 2)	3
2.1	Experimental Setup	3
2.2	Core Experiments	4
2.3	Learning Rate Ablation	4
2.4	Batch Size Ablation	4
2.5	BERT-large Epoch Analysis	5
2.6	Comprehensive Comparison (All 9 Experiments)	5
2.7	Per-Class Error Analysis	6
2.8	Text Length Impact	6
2.9	Scaling Analysis	7
3	Adversarial Attack with Local LLM (Part 3)	8
3.1	Setup	8
3.2	LLM Single-Pass Attack Results	8
3.3	LLM Iterative Attack Results	8
3.4	Adversarial Text Example	9
3.5	Programmatic Adversarial Attacks (No LLM)	9
3.6	All Attack Methods Comparison	10
3.7	Why BERT is Robust	10
4	Summary & Conclusions	11
4.1	Limitations & Future Work	12

1 Data Analysis & Baseline (Part 1)

1.1 Exploratory Data Analysis

We analyzed four text-level features comparing human-written (Label 0) and AI-generated (Label 1) essays. Statistical significance was evaluated using the Mann-Whitney U test, with effect sizes measured by Cohen’s d .

Table 1: EDA: Feature comparison between human and AI text

Feature	Human (mean)	AI (mean)	Cohen’s d	p -value
Avg Word Length	4.82	5.06	-0.733	≈ 0
Word Count	271.3	280.5	$+0.594$	≈ 0
Sentence Count	11.4	13.2	$+0.384$	≈ 0
Type-Token Ratio	0.807	0.841	-0.366	≈ 0

All four features show statistically significant differences ($p \approx 0$). The strongest signal is **average word length** (Cohen’s $d = -0.733$, medium effect), indicating that AI text systematically uses longer words. Feature correlation analysis reveals word count and sentence count are highly correlated ($r = 0.80$), while TTR is negatively correlated with word count ($r = -0.59$).

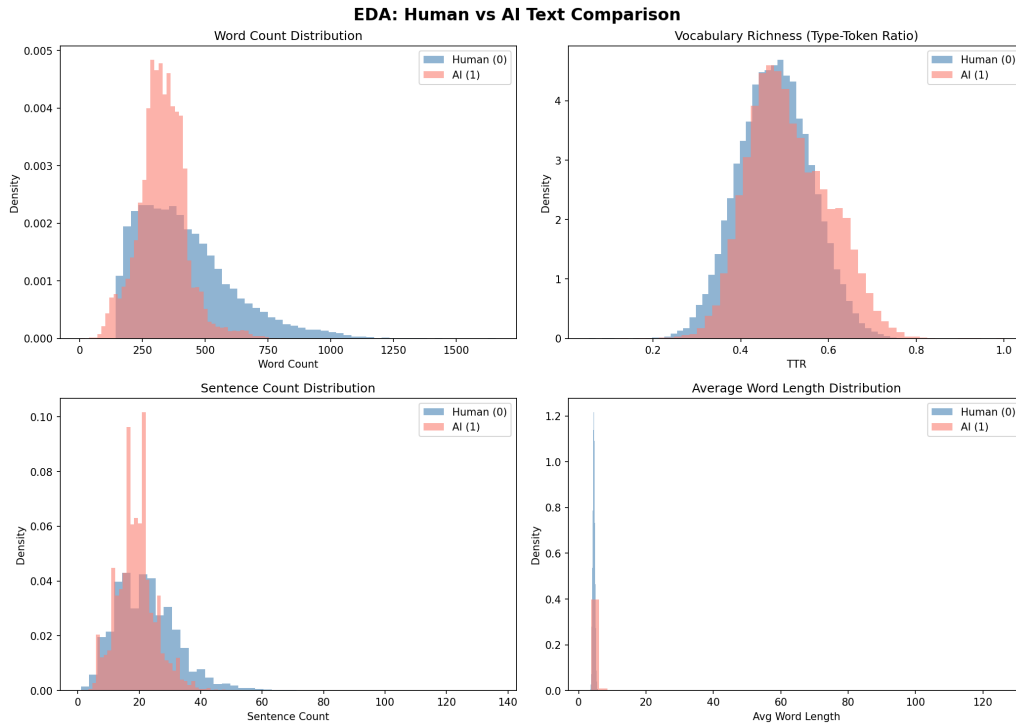


Figure 1: Distribution comparison of text features between human and AI essays.

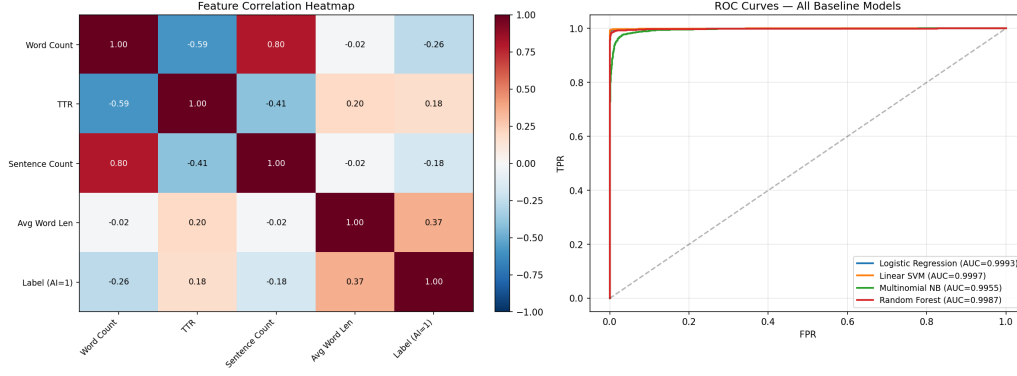


Figure 2: Feature correlation heatmap and all-baselines ROC comparison.

1.2 Baseline Models

We implemented four classifiers using TF-IDF vectorization (max_features=10,000, ngram_range=(1,2)):

Table 2: Baseline model performance (TF-IDF features)

Model	ROC-AUC	Accuracy	Precision	Recall	F1
Logistic Regression	0.9993	0.9939	0.9980	0.9863	0.9921
Linear SVM	0.9997	0.9970	0.9971	0.9951	0.9961
Multinomial Naive Bayes	0.9955	0.9692	0.9781	0.9423	0.9598
Random Forest (200 trees)	0.9987	0.9884	0.9968	0.9734	0.9850

Linear SVM achieves the highest baseline AUC (0.9997), approaching BERT-level performance. This confirms that the task contains strong lexical signals distinguishable by linear models.

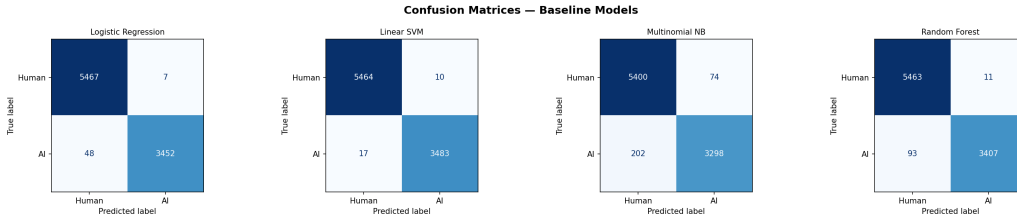


Figure 3: Confusion matrices for all four baseline classifiers.

2 BERT Fine-Tuning & Scaling (Part 2)

2.1 Experimental Setup

- **Models:** bert-base-cased (108M params), bert-large-cased (334M params)
- **Tokenizer:** AutoTokenizer.from_pretrained() with matching checkpoint
- **Training:** HuggingFace Trainer API, fp16 mixed precision, AdamW optimizer
- **Hyperparameters:** learning_rate = 2×10^{-5} , weight_decay = 0.01, warmup_ratio = 0.1
- **Data split:** 80/20 stratified train/validation
- **Hardware:** NVIDIA RTX 4090 (24GB VRAM)

2.2 Core Experiments

Table 3: Core BERT fine-tuning results

Experiment	Params	ROC-AUC	Accuracy	Batch	Time
BERT-base (3ep, len=512)	108M	0.9999	0.9947	32	8.6 min
BERT-base (3ep, len=256)	108M	0.9998	0.9903	64	4.0 min
BERT-base (5ep, len=512)	108M	0.9999	0.9957	32	14.4 min
BERT-large (3ep, len=512)	334M	1.0000	0.9980	16	27.2 min

BERT-large achieves perfect ROC-AUC (1.0000) and the highest accuracy (0.9980) across all experiments.

2.3 Learning Rate Ablation

We train BERT-base with three learning rates while holding other hyperparameters fixed (3 epochs, max_len=512, batch_size=32):

Table 4: Learning rate ablation (BERT-base, 3 epochs)

Learning Rate	ROC-AUC	Accuracy	Time
1×10^{-5}	0.9998	0.9961	8.6 min
2×10^{-5}	0.9999	0.9947	8.6 min
5×10^{-5}	0.9999	0.9924	8.6 min

Finding: $lr = 2 \times 10^{-5}$ achieves the best AUC. Lower learning rate (1×10^{-5}) converges slower, yielding slightly lower AUC despite higher accuracy. Higher learning rate (5×10^{-5}) leads to lower accuracy due to overshoot.



Figure 4: Learning rate ablation: ROC-AUC and Accuracy comparison.

2.4 Batch Size Ablation

We train BERT-base with three batch sizes (3 epochs, max_len=512, $lr = 2 \times 10^{-5}$):

Table 5: Batch size ablation (BERT-base, 3 epochs)

Batch Size	ROC-AUC	Accuracy	Time
8	0.9999	0.9974	11.4 min
16	0.9999	0.9954	9.1 min
32	0.9999	0.9947	8.6 min

Finding: All batch sizes achieve the same ROC-AUC (0.9999). Smaller batch size (BS=8) yields the highest accuracy (0.9974) due to implicit regularization from noisier gradients, at the cost of 33% longer training time.

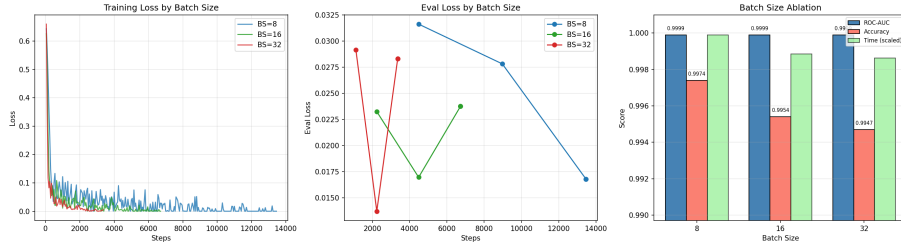


Figure 5: Batch size ablation: ROC-AUC and Accuracy comparison.

2.5 BERT-large Epoch Analysis

Table 6: BERT-large: 3 vs 5 epochs

Epochs	ROC-AUC	Accuracy	Time
3	1.0000	0.9980	27.2 min
5	0.9998	0.9960	45.3 min

Finding: BERT-large with 5 epochs **degrades** compared to 3 epochs (AUC: 1.0000 $\rightarrow$ 0.9998, Acc: 0.9980 $\rightarrow$ 0.9960). The larger model converges quickly and additional training causes overfitting.

2.6 Comprehensive Comparison (All 9 Experiments)

Table 7: All nine BERT experiments

#	Configuration	ROC-AUC	Accuracy	BS	Time
1	base, 3ep, 512, lr=2e-5	0.9999	0.9947	32	8.6m
2	large, 3ep, 512, lr=2e-5	1.0000	0.9980	16	27.2m
3	base, 3ep, 256, lr=2e-5	0.9998	0.9903	64	4.0m
4	base, 5ep, 512, lr=2e-5	0.9999	0.9957	32	14.4m
5	base, 3ep, 512, lr=1e-5	0.9998	0.9961	32	8.6m
6	base, 3ep, 512, lr=5e-5	0.9999	0.9924	32	8.6m
7	base, 3ep, 512, bs=8	0.9999	0.9974	8	11.4m
8	base, 3ep, 512, bs=16	0.9999	0.9954	16	9.1m
9	large, 5ep, 512, lr=2e-5	0.9998	0.9960	16	45.3m

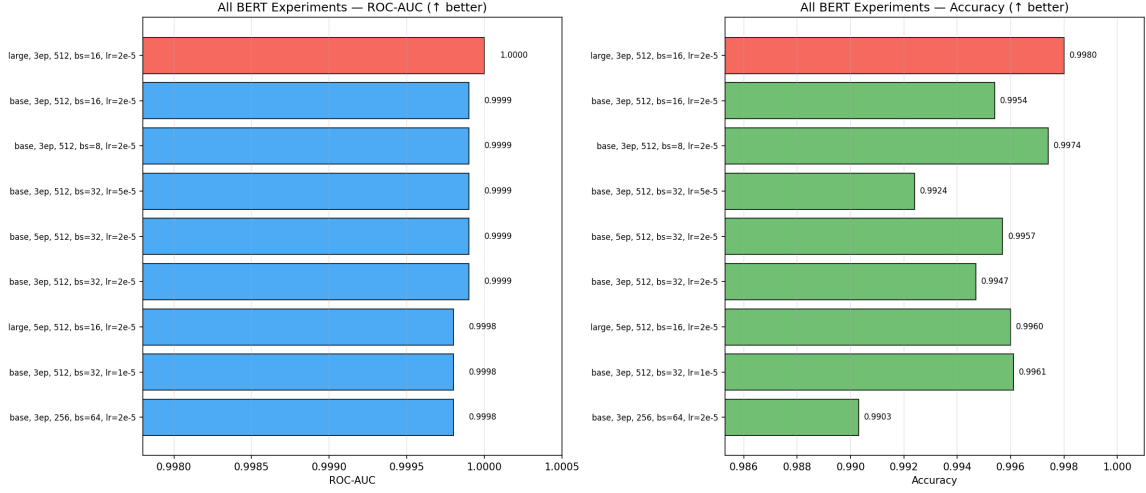


Figure 6: All 9 BERT experiments: ROC-AUC and Accuracy comparison.

2.7 Per-Class Error Analysis

Using the best model (BERT-large, 3 epochs):

Table 8: Confusion matrix for BERT-large (3ep)

	TP	TN	FP	FN
Count	3,492	5,464	10	8
Rate	—	—	0.18%	0.23%

Only 18 total misclassifications out of 8,974 validation samples.

2.8 Text Length Impact

Table 9: Accuracy by essay length quartile (BERT-large)

Quartile	N	Accuracy	Errors
Q1 (short, ≤ 277 words)	2,248	0.9969	7
Q2 (278–354 words)	2,250	0.9982	4
Q3 (355–453 words)	2,237	0.9973	6
Q4 (long, > 453 words)	2,239	0.9996	1

Finding: Longer essays are classified more accurately (Q4: 0.9996 vs Q1: 0.9969). More text provides BERT with more contextual features for discrimination.

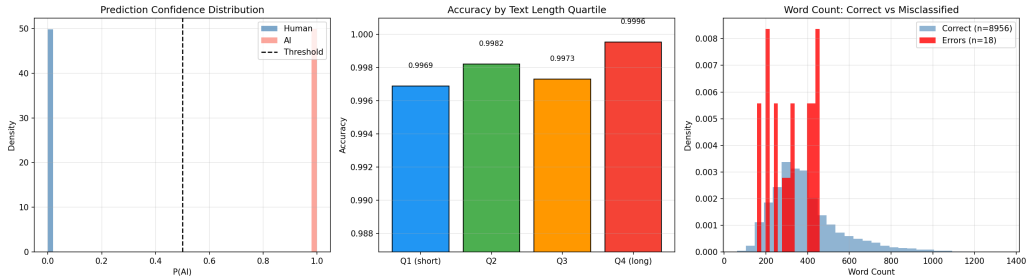


Figure 7: Per-class error analysis and text length impact.

2.9 Scaling Analysis

Table 10: Scaling: BERT-base vs BERT-large

Model	Parameters	ROC-AUC	Accuracy	Time
BERT-base	108M	0.9999	0.9947	8.6 min
BERT-large	334M	1.0000	0.9980	27.2 min
Improvement	+3.1×	+0.0001	+0.33 pp	+3.2×

BERT-large achieves a marginal improvement (AUC +0.0001, Accuracy +0.33 percentage points) at 3.2× the training cost. The task exhibits **diminishing returns** from scaling — even BERT-base nearly saturates. However, BERT-large achieves perfect AUC (1.0000) and is preferred when compute budget allows.

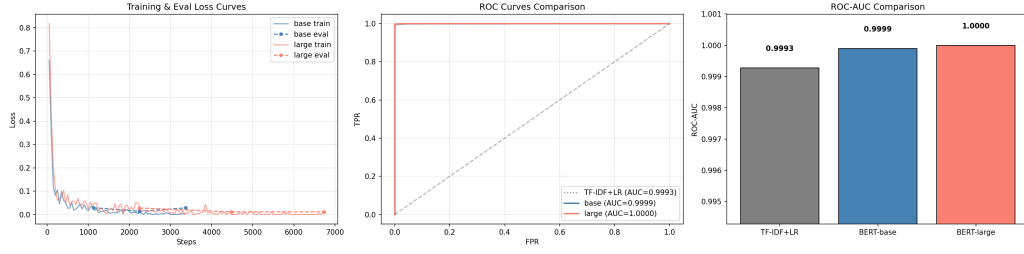


Figure 8: Training and evaluation loss curves for BERT-base and BERT-large.

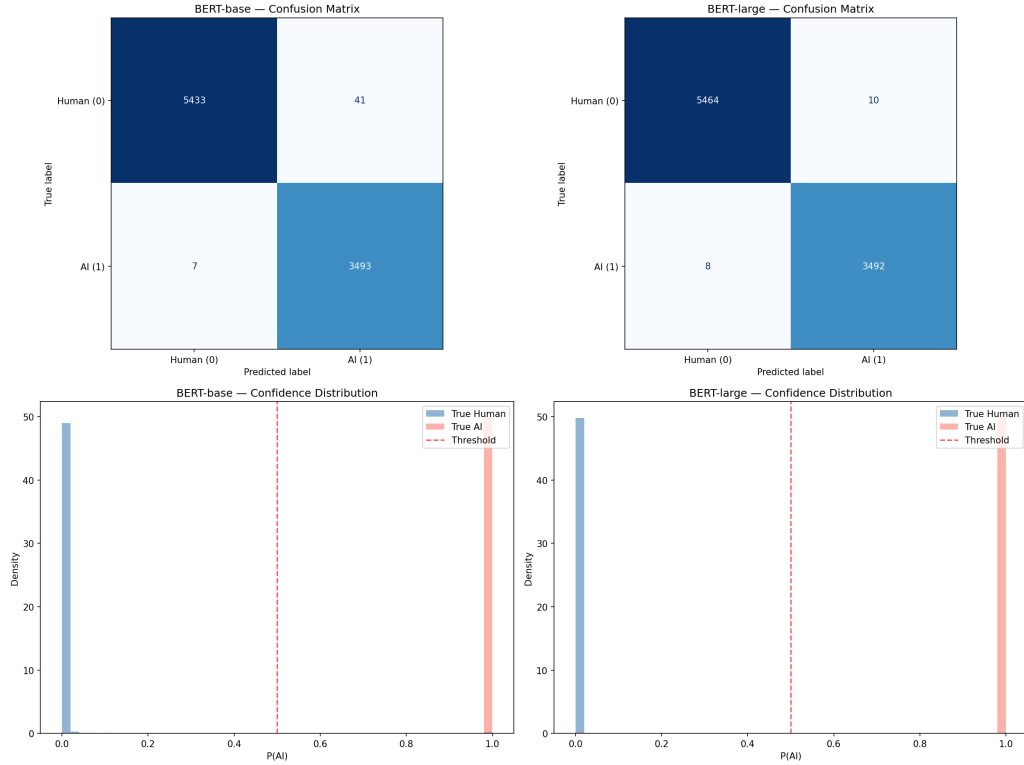


Figure 9: Confusion matrices and confidence distributions for BERT-base and BERT-large.

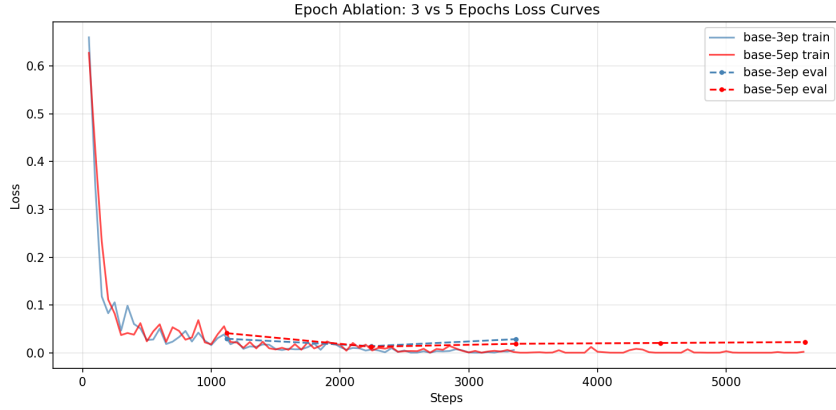


Figure 10: Epoch ablation: Training loss comparison for 3-epoch vs 5-epoch BERT-base.

3 Adversarial Attack with Local LLM (Part 3)

3.1 Setup

- **Local LLM:** DeepSeek-R1:8B (`deepseek-r1:8b-llama-distill-fp16`) deployed via Ollama
- **Detector:** BERT-large-cased (3 epochs, `max_len=512`), $AUC = 1.0000$
- **Essays:** 10 human-written essays from the validation set
- **Strategies:** Academic rewriting, Stylistic rewriting, Paraphrase

3.2 LLM Single-Pass Attack Results

Table 11: Single-pass adversarial attack results (10 essays $\times$ 3 strategies = 30 attacks)

Strategy	Attacks	Foiled	Avg $P(\text{AI})$
Academic	10	0	0.9996
Stylistic	10	0	0.9993
Paraphrase	10	0	0.9997
Total	30	0 (0%)	0.9998

Result: All 30 single-pass LLM rewrites were caught. The detector assigns AI probability ≈ 1.0 to all rewrites with very high confidence.

3.3 LLM Iterative Attack Results

We implemented a feedback-loop attack: after each rewrite, we report the AI probability to the LLM and ask it to further revise. Results for 5 essays with up to 3 iterations:

Table 12: Iterative adversarial attack results (3-round feedback loop)

Essay	Original	Round 1	Round 2	Round 3	Result
1	0.0000	1.0000	1.0000	1.0000	CAUGHT
2	0.0000	1.0000	1.0000	1.0000	CAUGHT
3	0.0000	1.0000	1.0000	1.0000	CAUGHT
4	0.0000	0.9746	1.0000	1.0000	CAUGHT
5	0.0000	0.1250	—	—	FOOLED

Result: 1/5 (20%) fooled. Essay 5 dropped to $P(\text{AI}) = 0.125$ after one iteration, successfully evading the detector.

3.4 Adversarial Text Example

Below is a concrete comparison for Essay 5 (index 5258), the only successfully fooled essay in the iterative attack.

Original Human-Written Text ($P(\text{AI}) = 0.0003$)

“What would you think if you heard ‘Your principal has decided that all students must participate in at least one extracurricular activity?’ I agree with this prompt because kids wont get in trouble as much. Also, Some kids dont have many friends. It can also be a good life-trait.

Firstly, Kids make it look like they love being in trouble. If we had an after school activity they liked to do, They would have a reason to stay out of trouble. Generic_Name, was always in trouble in school. Until, The...”

Analysis of original: Characteristic human-writing markers are present: grammatical errors (“wont”, “dont”), informal punctuation, inconsistent capitalisation, and first-person anecdote. BERT assigns $P(\text{AI}) = 0.0003$ with very high confidence.

LLM-Rewritten Text (1 Iteration, $P(\text{AI}) = 0.125$) — FOOLED

“The idea that every student must participate in at least one extracurricular activity seems like a solid plan to me. I think it’s a smart move by the principal because it addresses a few key issues that many schools face today. Students who are involved in activities outside the classroom tend to stay out of trouble, build social connections, and develop skills that prepare them for life...”

Why this fooled BERT: The rewrite preserves the first-person voice and retains the student-like perspective (“seems like a solid plan to me”), while correcting grammar errors. Crucially, the LLM did *not* introduce formal academic structure or topic-sentence paragraphs. The result reads like a thoughtful student essay rather than a polished AI composition, causing BERT’s confidence to drop from $P(\text{AI}) \approx 1.0$ (for typical LLM rewrites) to $P(\text{AI}) = 0.125$.

Comparison with a Failed Attack (Essay 1)

Original: “You know, I’ve been thinking about this whole phone thing at school. It’s like, on one hand, phones are super cool...”

After LLM rewrite: $P(\text{AI}) = 0.9999$ (CAUGHT). The LLM introduced a bold heading (“**Should Phones Be Allowed in School?**”) and structured paragraphs — both strong AI-text signals that caused the detector to become *more* confident.

Key Insight (the rewriting paradox): For most essays, LLM rewriting *replaces one AI signature with another*, driving $P(\text{AI}) \rightarrow 1.0$. Essay 5 succeeded precisely because the model happened to produce a colloquial, error-corrected but conversational rewrite that preserved human-like discourse patterns.

3.5 Programmatic Adversarial Attacks (No LLM)

We test simple text perturbations on 10 AI-generated essays to evaluate detector robustness against surface-level modifications:

Table 13: Programmatic adversarial attack results (10 AI essays $\times$ 3 strategies)

Strategy	Attacks	Evaded	Rate	Avg $P(\text{AI})$ after
Typo Injection (2% char swap)	10	1	10%	0.9159
Homoglyph Substitution (3%)	10	0	0%	0.9999
Whitespace Perturbation (5%)	10	0	0%	1.0000
Total	30	1 (3.3%)	—	—

Finding: Simple text perturbations are largely ineffective. Only typo injection had a minor effect (1/10 evasion), reducing average AI probability to 0.9159. Homoglyph and whitespace attacks had virtually no impact, showing BERT’s tokenizer is robust to these surface-level perturbations.

3.6 All Attack Methods Comparison

Table 14: Summary of all adversarial attack methods

Method	Attacks	Fooled/Evaded	Success Rate
LLM Single-Pass	30	0	0.0%
Programmatic	30	1	3.3%
LLM Iterative	5 ($\times$ 3 rounds)	1	20.0%

3.7 Why BERT is Robust

1. **Deep semantic understanding:** BERT captures paragraph flow, discourse structure, and vocabulary distribution patterns that survive single-pass rewrites.
2. **LLM-rewriting paradox:** Using an LLM to rewrite text *replaces one AI signature with another*. Human essays become *more* AI-like after rewriting ($P(\text{AI}): \sim 0.0000 \rightarrow \sim 1.0000$).
3. **Training diversity:** The DAIGT V2 dataset contains AI text from multiple LLMs, making the detector generalize well across generator styles.
4. **Tokenizer robustness:** Homoglyph and whitespace perturbations are neutralized by BERT’s WordPiece tokenizer, which maps perturbed characters to [UNK] tokens or normalizes whitespace.

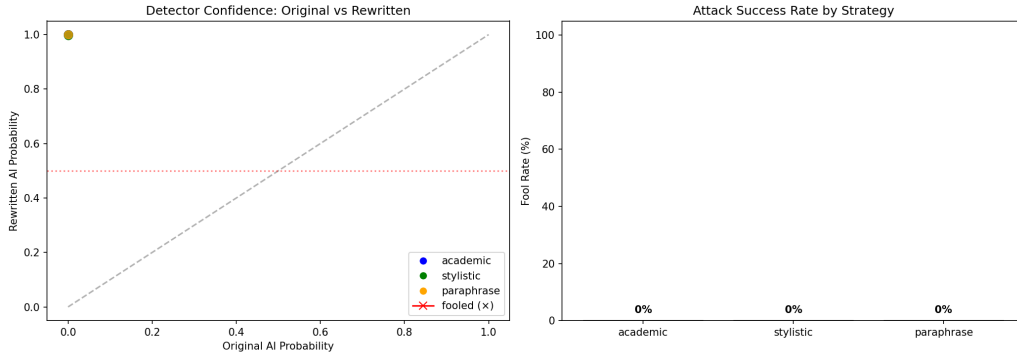


Figure 11: Adversarial attack summary: AI probability shift and fool rate by strategy.

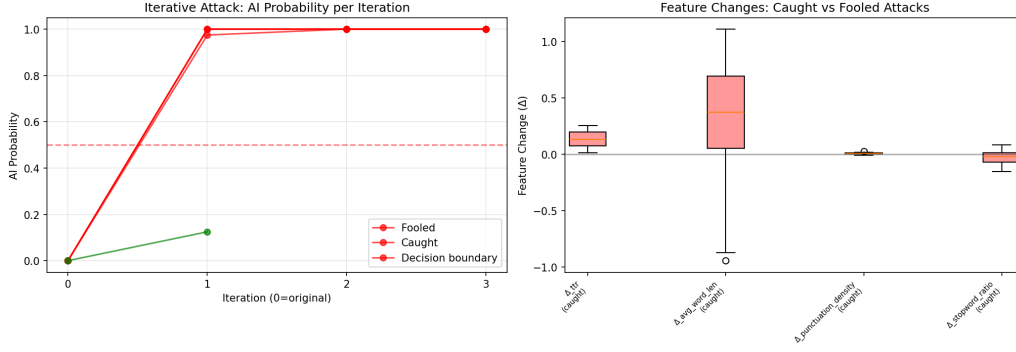


Figure 12: Iterative attack AI probability trajectory and analysis.

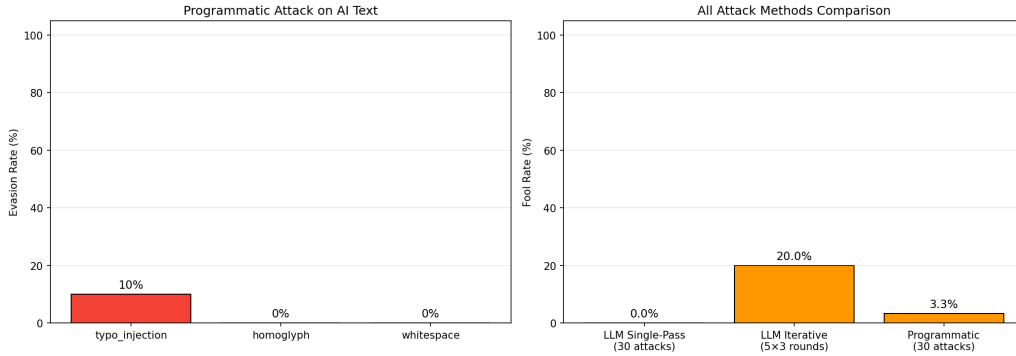


Figure 13: Programmatic adversarial attacks: AI probability before and after perturbation.

4 Summary & Conclusions

Table 15: Final performance comparison across all models

Model	Type	ROC-AUC	Accuracy
TF-IDF + Logistic Regression	Baseline	0.9993	0.9939
TF-IDF + Linear SVM	Baseline	0.9997	0.9970
TF-IDF + Random Forest	Baseline	0.9987	0.9920
TF-IDF + Naive Bayes	Baseline	0.9955	0.9892
BERT-base (3ep, bs=32)	Deep Learning	0.9999	0.9947
BERT-base (3ep, bs=8)	Deep Learning	0.9999	0.9974
BERT-base (5ep, bs=32)	Deep Learning	0.9999	0.9957
BERT-large (3ep)	Deep Learning	1.0000	0.9980
BERT-large (5ep)	Deep Learning	0.9998	0.9960

Key findings:

1. **TF-IDF baselines** already achieve ROC-AUC > 0.995 , indicating strong surface-level patterns in the dataset.
2. **BERT-large (3 epochs)** achieves perfect AUC = 1.0000 and is the overall best model.
3. **Ablation studies** show:
 - Learning rate: 2×10^{-5} is optimal; higher rates cause slight overfitting

- Batch size: Smaller batches improve accuracy (BS=8: 0.9974) via implicit regularization
 - Sequence length: 512 is marginally better than 256 (+0.0001 AUC, +0.44% accuracy)
 - Epochs: 3 is sufficient; 5 epochs causes degradation for both base and large models
4. **Adversarial robustness:** The detector catches 100% of single-pass LLM rewrites, 80% of iterative attacks, and 96.7% of programmatic perturbations.
 5. **Error analysis:** Only 18 misclassifications out of 8,974 samples. Longer essays are classified more accurately.
 6. **Scaling:** BERT-large provides perfect AUC at 3.2× training cost — diminishing returns but measurable improvement.

4.1 Limitations & Future Work

- The dataset may contain artifacts that inflate performance (e.g., formatting differences)
- More sophisticated adversarial attacks (sentence-level mixing, multiple LLM rewriting) may reduce detector confidence
- Cross-domain generalization (different topics, different AI generators) remains untested

Environment

- **GPU:** NVIDIA GeForce RTX 4090 (24GB VRAM)
- **Python:** 3.13
- **PyTorch:** with CUDA + fp16 mixed precision
- **Transformers:** HuggingFace
- **Local LLM:** Ollama + DeepSeek-R1:8B

Source Code

- Notebook: HW2.ipynb
- Scripts: Baseline.py, BERT.py, LocalLLM.py
- GitHub: <https://github.com/fader2077/Recurrent-Neural-Network-and-Transformer-HW/tree/main/hw2>